

UDAIPUR SCROLLBOOK

Technical System Design Report

A Collaborative Travel Photo Album Web Application

Table of Contents

1. Introduction
 2. System Overview
 3. Architecture and Backend Structure
 4. Authentication Flow
 5. Photo Upload Flow (Detailed)
 6. Data Storage Architecture
 7. Request Lifecycle
 8. API Endpoints Reference
 9. Design Decisions and Rationale
 10. Phase 2: Google Drive Archive
 11. Complete End-to-End Scenario
 12. Deployment and Infrastructure
 13. Summary
-

1. Introduction

Udaipur Scrollbook is a collaborative travel photo album application designed for groups of friends or family members to collectively upload and organize photos from shared trips. The core concept is simple: a user visits a landmark (such as City Palace or Lake Pichola in Udaipur), takes photos, and uploads them to the app. Those photos are then organized by location (place), creating a scrollable, visual timeline of the entire trip.

This report explains the entire system—from frontend user interactions through backend processing, data storage, image hosting, and authentication—in a way that makes sense to both beginners and experienced developers. It emphasizes the flow of information, the role each component plays, and the reasoning behind key design decisions.

1.1 Key Goals

- Enable multiple users to collaboratively upload photos from shared trips
- Organize photos by place (landmark or location)

- Provide smooth, scroll-based interface for browsing the album
- Handle user authentication securely
- Store photos efficiently using third-party cloud storage (Cloudinary)

1.2 Technology Stack

Category	Technology
Frontend	React 18 + TypeScript + Vite + React Query + Tailwind CSS
Backend	Node.js + Express + TypeScript
Database	MongoDB + Mongoose
Image Storage	Cloudinary (Phase 1), Google Drive (Phase 2)
Authentication	Google Sign-In + JWT (JSON Web Tokens)

2. System Overview

Before diving into technical details, let's understand the big picture: who interacts with the system, what roles each component plays, and how information flows from one part to another.

2.1 System Actors and Components

End User (Browser): A person accessing the Udaipur Scrollbook web app. They can log in, view photos grouped by place, and upload new photos. The user interacts with the system solely through the browser interface.

Frontend Application: The React app running in the user's browser. It handles user interactions (clicks, form submissions), displays photos and places, manages authentication locally, and communicates with the backend via HTTP requests.

Backend Server: A Node.js + Express application running on a server. It receives requests from the frontend, processes them (authentication checks, database queries, image upload coordination), and sends responses back. The backend is the single source of truth for business logic.

Database (MongoDB): Stores all structured data: user information, places, photo metadata (URLs, descriptions, upload date), and relationships between photos and places. The backend queries and updates this database.

Image Storage (Cloudinary): A third-party cloud service that stores the actual image files. It provides URLs for images, handles resizing, optimization, and delivery. The database

stores only the URLs, not the image files themselves.

Google Authentication (OAuth 2.0): A trusted third-party service that verifies user identity. Users log in using their Google account, and we use Google's tokens to verify their identity securely.

2.2 High-Level Data Flow

Here's a storytelling view of how data flows through the system when a user uploads a photo:

1. User selects a photo in the browser and chooses a place to associate it with.
 2. Frontend sends the image file and metadata to the backend via an HTTP POST request.
 3. Backend receives the request, checks the user's JWT token (is this user authenticated?), then uploads the image file to Cloudinary.
 4. Cloudinary returns a URL pointing to the hosted image.
 5. Backend saves a photo record in MongoDB with the Cloudinary URL, the place ID, user ID, and timestamp.
 6. Backend responds to the frontend with the new photo data.
 7. Frontend updates the UI to display the new photo in the correct place group.
-

3. Architecture and Backend Structure

This section explains the structure of the system and how each layer interacts with the others. Think of the system as a three-layer sandwich: frontend, backend, and data.

3.1 Layered Architecture

Presentation Layer (Frontend): React components that the user sees and interacts with. This layer is responsible for rendering UI, collecting user input, validating form fields, and displaying data. It does NOT contain business logic.

API Layer (Backend): Express routes and controllers that receive HTTP requests from the frontend and respond with data or confirmation. The API layer is the contract between frontend and backend—it defines what endpoints exist and what data they accept/return.

Business Logic Layer (Backend Services): Functions that handle the core logic: validating authentication, checking permissions, coordinating image uploads, saving data to the database. This layer is independent of HTTP—it could be reused by a CLI tool, a Discord bot, or any other client.

Data Layer (MongoDB + Cloudinary): Persistent storage. MongoDB stores metadata (who uploaded what, when, to which place), and Cloudinary stores the actual image files.

3.2 Backend Folder Structure and Purpose

Understanding the backend folder organization helps you find where to add features or fix bugs:

src/config/ Configuration files: database connection strings, environment variables, API keys for external services (Cloudinary, Google OAuth). This is where setup happens once, at startup.

src/models/ Mongoose schemas that define the structure of data in MongoDB. For example, a Photo model describes what fields a photo has (URL, placeId, userId, caption, etc.). These are blueprints for data.

src/controllers/ Functions that handle HTTP requests and responses. A controller receives the request, calls a service function to do the work, and formats the response. Example: `uploadPhotoController` receives a POST request, calls `uploadPhotoService`, then sends back the response.

src/middlewares/ Functions that run **before** a controller. Common middleware: authentication (verifying JWT), error handling, logging. If a middleware rejects a request, the controller never runs.

src/routes/ Express route definitions that map URLs to controllers. Example: `POST /api/photos → use authMiddleware, then call uploadPhotoController`. This is the routing layer.

src/services/ Business logic functions. Controllers call these to do the actual work (save to database, upload to Cloudinary). Services are independent of HTTP, making them testable and reusable.

src/utls/ Helper functions used across the application: token generation, error formatting, validation helpers. These are utility functions with no business logic.

src/app.ts The Express app setup file. It configures middleware globally, registers all routes, sets up error handling.

src/server.ts The entry point. It starts the server (listens on a port) and initializes the database connection.

4. Authentication Flow

Authentication is how we verify that a user is who they claim to be. This system uses a two-step approach: Google sign-in (identity verification) followed by JWT (session management).

4.1 Why Google Sign-In?

We use Google Sign-In instead of a custom login system because:

- Google is trusted and secure
- Users don't need to create or remember passwords
- We avoid storing sensitive password data
- Google handles all the security complexity for us

4.2 Why JWT (JSON Web Tokens)?

After Google verifies a user's identity, we create a JWT token. This token is a signed credential that the frontend includes in every request. The backend verifies the token's signature and expiration without needing to query a database, making it fast and scalable.

4.3 Step-by-Step Authentication Flow

Here's the complete authentication journey:

1. User opens the app in the browser. The frontend detects no JWT token in local storage (new user or session expired).
2. Frontend displays a "Login with Google" button. User clicks it.
3. Google's OAuth popup opens. User selects their Google account and grants the app permission to access their email and profile.
4. Google returns an **ID token** (a cryptographically signed claim that proves the user is who they say they are). The frontend receives this token.
5. Frontend sends the ID token to the backend's `POST /api/auth/google` endpoint.
6. Backend verifies the ID token's signature using Google's public key. If it's valid, the backend extracts the user's email.
7. Backend checks if a user with this email already exists in MongoDB. If not, it creates a new user record with the email.
8. Backend generates a new JWT token containing the user's ID and email. This token expires in 7 days (or a custom duration).
9. Backend responds with the JWT token. Frontend receives it and stores it in local storage.
10. Frontend now includes this JWT token in the Authorization header of every subsequent request.
11. When the token expires, the frontend detects the 401 response from the backend and triggers a new login flow.

4.4 Authenticating Subsequent Requests

Once logged in, the user wants to upload photos or view their profile. Here's how each request is authenticated:

Frontend makes a request (e.g., POST /api/photos) and includes the JWT in the Authorization header:

```
Authorization: Bearer <JWT_TOKEN>
```

1. Backend's authentication middleware intercepts the request before the controller runs.
 2. Middleware extracts the JWT from the Authorization header and verifies its signature and expiration.
 3. If valid, middleware attaches the decoded user data (user ID, email) to the request object so the controller can access it.
 4. If invalid or missing, middleware responds with a 401 Unauthorized error, and the controller never runs.
 5. Controller proceeds with the business logic, knowing the user is authenticated.
-

5. Photo Upload Flow (Detailed)

Uploading a photo is the core feature of the app. This section breaks it down into stages: frontend actions, backend processing, external service calls, and database updates.

5.1 Stage 1: User Action (Frontend)

1. User navigates to the "Upload Photo" section in the app.
2. Frontend displays a form with fields: (1) place selector (dropdown or list), (2) file input, (3) optional caption/description.
3. User selects a photo file from their device and optionally enters a caption.
4. User selects the place where the photo was taken (e.g., "City Palace").
5. User clicks "Upload" button.

5.2 Stage 2: Frontend Preparation

1. Frontend validates the form: Is a file selected? Is the file a valid image? Does it exceed size limits?
2. If validation fails, frontend shows an error message to the user. Process stops.
3. Frontend shows a loading spinner to indicate upload is in progress.
4. Frontend prepares a multipart form-data request containing: (1) the image file, (2) place ID, (3) caption, (4) JWT token in the Authorization header.
5. Frontend sends the request to `POST /api/photos` on the backend.

5.3 Stage 3: Backend Authentication

1. Request arrives at the backend. Express middleware extracts the JWT token from the Authorization header.
2. Middleware verifies the JWT: Is the signature valid? Has it expired? If not, the request is rejected with 401 Unauthorized.
3. If valid, middleware decodes the token and attaches user information (userId, email) to the request object.
4. Request proceeds to the upload controller.

5.4 Stage 4: Backend Controller Processing

1. Controller receives the request with the image file, place ID, caption, and authenticated user ID.
2. Controller performs server-side validation: Is the file actually an image? Does it exceed size limits? Does the place exist?
3. If validation fails, controller responds with a 400 Bad Request error and a descriptive message.
4. Controller calls the photo upload service, passing the file, place ID, caption, and user ID.

5.5 Stage 5: Service Layer (Upload to Cloudinary)

1. Service function receives the image file. It reads the file into memory.
2. Service makes an HTTP request to Cloudinary's API, uploading the image file. It includes Cloudinary API credentials (API key and secret) for authentication.
3. Cloudinary processes the image: optimizes it, stores it on their servers, generates a unique public URL.
4. Cloudinary responds with metadata: the image URL, dimensions, file size, public ID (Cloudinary's unique identifier).
5. Service receives the Cloudinary response and extracts the image URL.

5.6 Stage 6: Database Storage

1. Service creates a new photo record in MongoDB with fields: (1) URL from Cloudinary, (2) place ID, (3) user ID, (4) caption, (5) timestamp, (6) Cloudinary public ID (for future deletion/updates).
2. MongoDB saves the record and returns the complete photo object (including MongoDB's auto-generated ID).
3. Service returns the photo object to the controller.

5.7 Stage 7: Backend Response

1. Controller receives the photo object from the service.
2. Controller formats the response as JSON and responds with HTTP 201 (Created) status code, including the full photo object.
3. Frontend receives the response.

5.8 Stage 8: Frontend UI Update

1. Frontend detects the successful 201 response and extracts the photo object.
2. Frontend updates React state to include the new photo in the correct place's photo list.
3. React re-renders the component tree. The new photo appears in the scroll feed under its place.
4. Frontend hides the loading spinner and shows a success message to the user.
5. User sees their uploaded photo in the app.

5.9 Why This Multi-Stage Approach?

Each stage has a specific purpose:

- **Frontend validation** ensures the user doesn't send invalid data (poor user experience).
 - **Authentication middleware** prevents unauthorized uploads (security).
 - **Server-side validation** prevents malicious data (security).
 - **Cloudinary** offloads image storage burden from our server (scalability).
 - **Database record keeping** allows us to query photos by place, user, or date (usability).
 - **Explicit response back to frontend** keeps the user informed and allows UI optimization.
-

6. Data Storage Architecture

Data is stored in multiple places for different reasons. Understanding where each piece of data lives and why is crucial to system design.

6.1 MongoDB Collections

MongoDB stores all structured, queryable data. Think of collections as tables in a spreadsheet:

Users Collection: Stores user profiles. Fields: `userId`, `email`, `name` (from Google), `createdAt`. One record per user.

Places Collection: Stores landmark information. Fields: name (e.g., "City Palace"), description, placeSlug (unique identifier for URLs), coordinates (latitude/longitude), createdAt. One record per place.

Photos Collection: Stores photo metadata. Fields: cloudinaryUrl, cloudinaryPublicId, placeId (reference to Places), userId (reference to Users), caption, uploadedAt. One record per uploaded photo.

6.2 Cloudinary Storage

Cloudinary is a cloud-based image hosting service. We don't download full resolution images to our server. Instead, we upload files to Cloudinary and store only the URL in MongoDB. Cloudinary handles image optimization, caching, and delivery. When a user's browser requests a photo, it fetches from Cloudinary's CDN, not our server, which is faster and cheaper.

6.3 Frontend Local Storage

The browser's local storage (a key-value store) holds:

- JWT token for authentication
- Cached user profile (to avoid refetching on page reload)
- Cached list of places (since places rarely change)

This reduces backend queries and improves perceived performance.

6.4 Why Separate Metadata and Images?

You might ask: why not store images directly in MongoDB or on our server? Three reasons:

1. **Images are large.** Storing them in MongoDB would bloat the database and slow down queries.
2. **Serving images from our server consumes bandwidth and CPU.** Cloudinary's CDN is designed for fast image delivery globally.
3. **Cloudinary provides image optimization** (resizing, compression, format conversion) automatically. Our code doesn't need to handle this complexity.

6.5 Relationships Between Data

MongoDB relationships are established via IDs:

- Each Photo has a `placeId` field that references a Place record. To fetch all photos for City Palace, the backend queries: "find all photos where placeId = CityPalaceId".
- Each Photo has a `userId` field that references a User record. To fetch all photos a user uploaded, the backend queries: "find all photos where userId = UserId".
- This structure allows flexible querying without needing to join massive image files.

7. Request Lifecycle and Backend Architecture

Every HTTP request to the backend travels through multiple layers. Understanding this journey helps you know where to add code, where errors occur, and how to debug.

7.1 The Request Path

Imagine a request to upload a photo travels through this path:

1. Frontend sends: **POST /api/photos with JWT token and image file**
2. Express server receives the HTTP request
3. Express checks routing: Is there a handler for POST /api/photos? (Yes, in routes file)
4. Before the route handler runs, Express executes global middleware (like body parser, CORS headers)
5. Route-specific middleware runs (like authentication). If it rejects, the request stops here.
6. The controller function (uploadPhotoController) executes. It calls a service function.
7. The service function performs business logic: upload to Cloudinary, save to database.
8. Service returns result to controller
9. Controller formats response as JSON and sends HTTP 200/201 status
10. Frontend receives the response and updates the UI

7.2 Role of Each Layer

Routes (Express routes file): Define URL patterns (endpoints) and attach middleware/controllers to them. Think of it as a directory: "POST /api/photos → use authMiddleware, then call uploadPhotoController".

Middleware (middleware files): Functions that run before or after a route handler. Authentication middleware verifies JWT tokens. Error handling middleware catches exceptions. Each middleware can modify the request or reject it.

Controllers (controller files): Handle HTTP-specific logic: extracting data from the request, calling services, formatting responses. Controllers should be thin—they don't contain business logic, just orchestration.

Services (service files): Contain business logic. They are independent of HTTP: they take input parameters, perform calculations or database operations, return results. A service could be called by a controller, a cron job, or a CLI tool.

Models (model files): Define the structure of data in MongoDB. They specify what fields exist, what types they are, and any validation rules. When services interact with the database, they do so through models.

7.3 Error Handling

Errors can occur at any layer:

- **Middleware rejects invalid JWT** → responds 401
 - **Controller fails form validation** → responds 400
 - **Service throws exception** (e.g., Cloudinary API fails) → catches it, logs it, returns error
 - **Global error handler middleware** catches unhandled exceptions and responds with 500
-

8. API Endpoints Reference

The backend exposes several endpoints that the frontend calls. Each endpoint is documented with its purpose, request format, and response format:

8.1 Google Authentication

Endpoint: POST /api/auth/google

Purpose: Authenticate a user with a Google ID token and return a JWT.

Request Body: `{ "idToken": "<Google ID token>" }`

Response (201): `{ "token": "<JWT token>", "user": { "id": "...", "email": "..." } }`

Errors: 400 (invalid token), 500 (server error)

8.2 Get Current User

Endpoint: GET /api/auth/me

Purpose: Retrieve the authenticated user's profile.

Headers: `Authorization: Bearer <JWT>`

Response (200): `{ "id": "...", "email": "...", "name": "..." }`

Errors: 401 (unauthorized)

8.3 Upload Photo

Endpoint: POST /api/photos

Purpose: Upload a new photo to a place.

Headers: `Authorization: Bearer <JWT>, Content-Type: multipart/form-data`

Request Body: Form data with fields: file (image), placeId, caption (optional)

Response (201): `{ "id": "...", "url": "...", "placeId": "...", "userId": "...", "caption": "...", "uploadedAt": "..." }`

Errors: 400 (invalid file), 401 (unauthorized), 500 (server error)

8.4 Get Photos by Place

Endpoint: GET /api/photos?placeSlug=city-palace

Purpose: Retrieve all photos for a specific place.

Query Parameters: placeSlug (required)

Response (200): `{ "place": {...}, "photos": [{...}, {...}] }`

Errors: 404 (place not found)

8.5 Get All Photos

Endpoint: GET /api/photos/all

Purpose: Retrieve all photos across all places.

Response (200): `[{...}, {...}]`

Errors: 500 (server error)

9. Design Decisions and Rationale

Every significant choice in the architecture has a reason. This section explains the "why" behind major decisions.

9.1 Why Cloudinary for Image Storage?

Option 1: Store images on our server.

Cons:

- Disk space fills quickly with many users
- Serving images consumes bandwidth (expensive)
- We must handle image resizing, compression, format conversion
- Slow for users far from our server

Option 2 (Chosen): Use Cloudinary.

Pros:

- Unlimited storage
- Global CDN for fast delivery
- Automatic image optimization

- Pay per usage (cheap at small scale)
- Handles all image operations (resize, crop, format conversion) via URL parameters
- Frees our team from image infrastructure work

Trade-off: Dependency on a third-party service. If Cloudinary goes down, we can't serve images.

Mitigation: Cloudinary is reliable and widely used; we could add fallback storage later if needed.

9.2 Why JWT Tokens?

Option 1: Session-based authentication (server stores session state).

Cons:

- Server must store every active session in memory or database
- Scales poorly (multiple servers need shared session storage)
- Adds database query overhead per request

Option 2 (Chosen): JWT tokens.

Pros:

- Stateless—server doesn't store tokens (scales infinitely)
- Token contains all needed info (user ID, expiration)
- Verification is fast (just signature check, no database query)
- Works across multiple servers (no session sharing needed)
- Safe when using HTTPS (tokens are cryptographically signed)

Trade-off: Revoking a token before expiration is harder (we can't instantly log out a user if needed).

Mitigation: We use a 7-day expiration; for critical security, we could maintain a token blacklist.

9.3 Why Google Sign-In?

Option 1: Custom email/password signup.

Cons:

- We must securely hash and store passwords
- Users must remember passwords (leads to weak passwords or forgetting)
- We must implement password reset
- Users distrust new auth systems (Google is trusted)

Option 2 (Chosen): Google Sign-In (OAuth 2.0).

Pros:

- We never handle passwords (no password security responsibility)
- Users already trust Google
- Users don't need another password
- Google's auth is more secure than most custom implementations
- No password reset code—just "sign in with Google"

Trade-off: Dependency on Google. If their service is down, users can't log in.

Mitigation: Google's infrastructure is extremely reliable.

9.4 Why placeSlug?

Places are identified by a "slug": a URL-friendly unique string like "city-palace" or "lake-pichola". Instead of using MongoDB's internal ID (e.g., "6708f3a1c2b4d5e9f2g8h9i0"), slugs are human-readable and don't expose internal system details.

Pros of slugs:

- URLs are readable: `/photos/city-palace` vs `/photos/6708f3a1c2b4d5e9f2g8h9i0`
- Slugs are stable (a place's slug doesn't change even if the internal ID changes)
- SEO-friendly (search engines prefer readable URLs)
- Easy to share links

Trade-off: We must ensure slugs are unique.

Mitigation: When creating a place, we enforce uniqueness at the database level.

9.5 Why MongoDB?

Option 1: Relational database (PostgreSQL).

Pros for structured data with clear relationships.

Cons: Requires upfront schema definition, rigid structure.

Option 2 (Chosen): MongoDB (NoSQL document database).

Pros:

- Flexible schema (easy to add fields later)
- Data is stored as JSON-like documents (matches our JavaScript frontend)
- Fast for simple queries
- Good for rapid development (schema can evolve)
- Scales horizontally (sharding)

Trade-off: No automatic enforcement of data consistency.

Mitigation: We use Mongoose (ODM) to define schemas and validation.

10. Phase 2: Google Drive Archive

Phase 1 uses Cloudinary, which is great for serving photos. Phase 2 will add long-term archival to Google Drive: all photos will also be backed up to the user's personal Google Drive for lasting storage.

10.1 Why Async Background Upload?

When a user uploads a photo in Phase 1, the backend does this: upload to Cloudinary → save to database → respond to frontend. All happens synchronously (one after another). The frontend waits for the full flow to complete before showing the uploaded photo.

In Phase 2, we'll add: upload to Cloudinary → save to database → respond to frontend → (separately, in the background) upload to Google Drive. We respond to the frontend immediately after the database save, without waiting for Google Drive. Why?

- **Google Drive upload is slow** (network latency). Users don't want to wait extra seconds for a backup operation.
- **The photo is already stored** in Cloudinary and the database. Google Drive is just an extra safety net.
- **If Google Drive fails**, the photo is still safe and accessible. Users can still see and share it.

10.2 Implementation Overview

1. User uploads a photo. Backend follows Phase 1 flow: Cloudinary → database → respond.
2. After responding, backend enqueues a background task: "upload this photo to Google Drive".
3. A background job worker picks up the task asynchronously (seconds or minutes later, it doesn't matter).
4. Worker authenticates with Google Drive using OAuth, creates a folder for the trip, and uploads the photo.
5. Worker updates the photo record in MongoDB with the Google Drive file ID (for future reference/deletion).

10.3 Technology for Background Jobs

Options: (1) Bull (Node.js task queue), (2) RabbitMQ (message broker), (3) AWS SQS. We'll

likely use Bull, which is simpler for small scale and works well with Node.js + Redis.

Flow: Backend enqueues a job → Bull stores it in Redis → Worker process continuously pulls jobs from Redis → processes them → marks as complete.

Benefit: Upload to Cloundinary and save to database are fast (< 5 seconds). Google Drive upload happens in the background and doesn't block the user's experience.

11. Complete End-to-End Scenario

Let's trace a complete user journey to tie everything together:

Scenario: Friends are visiting Udaipur. Alice uploads a photo of City Palace, writes "Beautiful sunset!" as the caption.

11.1 Alice Starts the App

1. Alice opens the web app in her browser.
2. Frontend checks if a JWT token exists in local storage. It doesn't (first time or session expired).
3. Frontend displays a login screen with "Sign in with Google" button.

11.2 Alice Logs In

1. Alice clicks "Sign in with Google". Google's OAuth popup opens.
2. Alice selects her Google account. Google responds with an ID token (a signed claim proving her identity).
3. Frontend sends the ID token to `POST /api/auth/google`.
4. Backend verifies the token (signature, expiration), extracts Alice's email (`alice@gmail.com`).
5. Backend checks MongoDB: does a user with `alice@gmail.com` exist? No (first time).
6. Backend creates a new user record in MongoDB: `{ email: "alice@gmail.com", createdAt: now }`.
7. Backend generates a JWT token containing Alice's user ID and email, expiring in 7 days.
8. Backend responds with the JWT. Frontend stores it in local storage.
9. Frontend now shows the main app: a list of places (City Palace, Lake Pichola, etc.) and an upload button.

11.3 Alice Navigates to City Palace

1. Alice scrolls the app and clicks on "City Palace" place card.

2. Frontend makes a request: `GET /api/photos?placeSlug=city-palace` with the JWT in the Authorization header.
3. Backend's authentication middleware verifies the JWT. Valid. Proceeds.
4. Backend's controller queries MongoDB: "find the place with slug = city-palace and all photos for this place."
5. MongoDB returns the place record and an array of photos (currently empty; no one has uploaded yet).
6. Backend responds with: `{ "place": { "name": "City Palace", ... }, "photos": [] }`.
7. Frontend displays the City Palace page with an empty photo list and an "Upload Photo" button.

11.4 Alice Uploads a Photo

1. Alice clicks "Upload Photo". A form appears: file input, caption field.
2. Alice selects a photo from her phone and types "Beautiful sunset!" in the caption field.
3. Alice clicks "Upload". Frontend validates: file is an image (yes), size is under 10MB (yes).
4. Frontend shows a loading spinner. It sends: `POST /api/photos` with form data (file, placeId, caption, JWT in header).
5. Backend's authentication middleware verifies JWT. Valid. Attaches Alice's user ID to the request.
6. Controller validates: file is an image (yes), place exists (yes). Calls `uploadPhotoService`.
7. Service reads the image file and uploads it to Cloudinary. Cloudinary responds with a URL: `https://res.cloudinary.com/.../image123.jpg`.
8. Service saves a photo record to MongoDB: `{ url, placeId, userId, caption: "Beautiful sunset!", uploadedAt: now }`.
9. Service returns the complete photo record to the controller.
10. Controller responds with HTTP 201 (Created) and the photo record.
11. (Phase 2 only) Backend enqueues a background job: "upload this photo to Google Drive." Returns immediately.
12. Frontend receives the 201 response. It updates React state: adds the photo to the City Palace photo list. React re-renders. Alice sees her photo in the feed with the caption "Beautiful sunset!" and a loading indicator (indicating the Google Drive backup is pending). The spinner disappears.

11.5 Friends See Alice's Photo

1. Bob and Carol (other friends) also visit the app. They log in via Google.
 2. They navigate to City Palace.
 3. Backend queries: get all photos for City Palace. MongoDB returns Alice's photo.
 4. Bob and Carol see Alice's photo on their screens. They can click it to view the full image (loaded from Cloudinary's CDN). They can also upload their own photos.
-

12. Deployment and Infrastructure

Once development is complete, the app needs to run somewhere for real users. Here's an overview of deployment considerations:

12.1 Frontend Deployment

The React app (built with Vite) is compiled into static files (HTML, CSS, JavaScript). These are deployed to a CDN (Content Delivery Network) like Vercel, Netlify, or AWS CloudFront. Benefit: static files are served from locations near the user (fast). No server computation needed.

Configuration: Frontend must know the backend's API URL. Store this in an environment variable so it can be different for development (`http://localhost:3000`) vs. production (`https://api.udaipurscrollbook.com`).

12.2 Backend Deployment

The Node.js server runs on infrastructure like Heroku, AWS EC2, DigitalOcean, or Railway. The server is always running, listening for HTTP requests, connecting to MongoDB, and calling external services (Cloudinary, Google).

Configuration: Environment variables store sensitive data: MongoDB connection string, Google OAuth credentials, Cloudinary API key. These should never be in code or version control.

Monitoring: Use logging (e.g., Winston, Pino) to record errors and important events. Monitor uptime and response times. Set up alerts for crashes.

12.3 Database Deployment

MongoDB can be self-hosted (run it yourself) or use MongoDB Atlas (managed service). Atlas is recommended: they handle backups, scaling, security, and uptime. Backend connects to MongoDB via a connection string (stored as an environment variable).

Backups: Set up automated backups. Restore from backups if data is corrupted or accidentally deleted.

12.4 External Services

Cloudinary: Sign up for a free account (or paid if storage exceeds free limits). Store the API key and secret as environment variables. If considering Phase 2, set up a Google Cloud project for OAuth and Google Drive API.

HTTPS: Ensure all communication (frontend ↔ backend ↔ external services) is encrypted with HTTPS. Get an SSL certificate (free from Let's Encrypt) and enforce HTTPS redirects.

13. Summary

Udaipur Scrollbook is a well-architected system that balances simplicity, scalability, and maintainability:

- **Frontend (React):** Provides a responsive, user-friendly interface. Handles user input, validation, and displays data.
- **Backend (Node.js + Express):** Processes requests, orchestrates business logic, manages authentication, and coordinates with external services.
- **Database (MongoDB):** Stores all structured data in a flexible, scalable manner.
- **Image Storage (Cloudinary):** Handles image hosting, optimization, and CDN delivery without burdening our infrastructure.
- **Authentication (Google OAuth + JWT):** Ensures secure, user-friendly login without handling passwords.
- **Phase 2 (Google Drive Async Backup):** Adds long-term archival without impacting user experience.

Every architectural decision serves a purpose: security, scalability, simplicity, or user experience. As the system grows and requirements evolve, this design provides a solid foundation for adding new features.

Next Steps for Implementation

For developers building this system, the next steps are:

1. Set up the development environment (Node.js, MongoDB, Cloudinary account)
2. Implement the folder structure described in Section 3.2
3. Build out the models (User, Place, Photo)
4. Implement authentication flows
5. Implement the photo upload feature
6. Build the frontend UI to tie it all together

This report provides the complete system blueprint. Use it as a reference when implementing, debugging, or extending the application.

End of Report